Construction Scheduling Feature Blueprint (mybidbook.com)

This document outlines the architecture, database mechanics, workflow rules, and user interface specifications for adding an automated, dependent-path construction scheduling feature to won projects on **mybidbook.com**.

1. Feature Entry Constraints & Navigation

Left Menu Navigation

- Add a new menu option titled **"Scheduling"** on the left-side main navigation panel.
- This option must be accessible to users who have permissions to manage jobs and view quotes.

Entry Criteria

- The scheduling feature is strictly gated. A schedule record cannot be created, modified, or viewed for any project unless its associated quote is officially marked with a state of **"Won"**.
- Quotes in draft, sent, or lost status must not appear in the scheduling dashboard.

2. Project Scheduling Statuses

Each project with a "Won" quote must have a dynamically calculated **Scheduling Status**. The system evaluates the state of the project's task list to assign one of the following five lifecycle stages:

Status Name	Logical Trigger / Criteria
Not Scheduled	The quote is marked "Won," but no schedule record or active tasks have been created for the project yet.
Partial Schedule	A schedule has been initiated, and some tasks exist, but the sequence is incomplete (e.g., missing mandatory template steps or some tasks lack duration/dependencies).
Scheduled	A complete sequence of tasks has been

	mapped out from start to finish. All tasks have defined durations and valid calculated dates, but **no work has begun** (no actual dates logged).
In Progress	Triggered immediately when **the first actual date** (either Actual Start Date or Actual Completion Date) is recorded on any task in the sequence.
Completed	Triggered when **all tasks** in the active schedule have an Actual Completion Date successfully recorded.

3. The Scheduling Dashboard (UI / UX)

Default View (Active Projects)

- By default, clicking "Scheduling" displays a table of all active "Won" projects.
- This list includes projects with statuses of **Not Scheduled**, **Partial Schedule**, **Scheduled**, and **In Progress**.
- The table should display:
  1. Project Name / Client Info
  2. Original Quote Number / Value
  3. Calculated Project Start Date & Projected Completion Date
  4. Current Scheduling Status (color-coded badges for clarity)
  5. Action Button (e.g., "Build Schedule" or "Manage Timeline")

Completed Projects View

- To prevent clutter, projects with a status of **Completed** are hidden from the default active list.
- **The Filter Toggle:** Provide a status filter control (such as a tab system or segment buttons: Active Jobs vs. Completed Jobs) at the top of the screen. This must respect the visual identity and function of how completed quotes are currently displayed on mybidbook.com.
- Clicking the Completed Jobs toggle loads the archive of finished projects with read-only schedule summaries.

4. Automated Transition & Completion Pop-up

Completion Event Trigger

The database or state manager must listen to updates on the project's task records.

1. When a user inputs the Actual Completion Date for the final outstanding task in the schedule:
    - The system detects that 100% of tasks now have actual end dates.
    - The project's overall scheduling status is set to **Completed**.
2. **The UI Alert:** On saving this final date, the interface must intercept the save action and display a stylized modal pop-up to the user.
    - *Do not use standard browser alerts.* Render a premium, custom UI modal.
    - **Modal Text:** *"Congratulations! All tasks on this project are now completed. This project is being transitioned to Completed and moved to your completed projects archive."*
    - **Action:** Clicking "Dismiss" or "OK" refreshes the table, successfully archiving the project out of the active scheduling tab.

5. Calendar Exceptions, Catch-Up Saturdays, & Inheritance

To determine realistic working timelines, the calculation engine needs to evaluate weekends, company holidays, and custom project-level exceptions.

The "Inherit and Override" Pattern

1. **Global Company Settings:** Define default working days (e.g., Mon–Fri) and standard company holidays globally.
2. **The Project Copy (Clone on Win):** The moment a quote transitions to "Won," create a localized calendar configuration for that project by taking a snapshot copy of the global company holiday settings.
3. **Project-Specific Adjustments:** Allow the user to modify this local copy for each project individually (e.g., Project A works on a specific holiday/weekend to catch up, while Project B remains closed).

The Day Evaluation Logic (isWorkingDay)

For any calendar date being processed by the scheduling engine, evaluate it using the following sequence:

// Chronological evaluation rules:
1. Is this date in the project's "Custom Workdays" list? (e.g. Catch-up Saturday)
  => YES: Return TRUE (It is a working day, skipping weekend checks)

2. Is this date in the project's "Custom Holidays / Off-Days" list?

=> YES: Return FALSE (It is a non-working day, skipping other checks)

3. Is this date in the cloned Company Holidays snapshot?
=> YES: Return FALSE

4. Fall back to standard weekly rules: Is this day of the week selected as a working day?
=> YES: Return TRUE
=> NO: Return FALSE

6. Cascading Schedule & Dependency Calculations

Whenever a task's duration changes, a dependency is modified, or a task is marked as "Firm" (Start No Earlier Than), the downstream schedule must automatically recalculate using a cascade loop.

Downstream Cascade Rule

For any given task ($Task_B$) that depends on ($Task_A$):

$$\text{Calculated Start Date}_B = \text{Target End Date}_A + \text{Lag Days}_B$$

Resolving "Target" Dates

- **If parent task has not started:** Use calculated dates.
- **If parent task is completed:** Set the predecessor's end date to its logged Actual End Date. Run downstream calculations from this actual day forward.
- **If parent task is late/overrun:** If the task is past its expected timeline and has an actual start date, project its finish date based on Actual Start Date + Duration, pushing downstream tasks out proportionately.
- **Constraint Rule (Firm Status):** If a task is toggled to **"Firm"** with a specific date, its operational start date must be the maximum of the calculated date or the manual firm date:

  $$\text{Target Start Date} = \text{MAX}(\text{Calculated Start Date}, \text{Manual Firm Date})$$

  This prevents upstream delays from silently scheduling successor tasks before their physical predecessors can finish, while allowing locks on future bookings.

eof

Suggested Adjustments & Implementation Order

When you are ready to begin building this with your coding AI, I suggest taking a **modular, step-by-step approach** to avoid introducing too many complex states at once. Here is a recommended road map to follow:

1. **Step 1: Database Setup (The AI-Schema Session)**
   First, feed the AI your current database schema (PostgreSQL, MySQL, Mongo, etc.) and ask it to write the migration script to add the relational tables for Project Schedules, Active Tasks, and Calendar Settings/Exceptions based on the conceptual descriptions in this blueprint.
2. **Step 2: The Core Date Engine**
   Have the AI write the pure JavaScript functions isWorkingDay and addWorkingDays first. Test these helper functions thoroughly with dummy variables to ensure that adding a 5-day duration to a Thursday correctly skips Saturdays, Sundays, and custom holidays.
3. **Step 3: The Left Navigation & List Views**
   Build out the layout. Configure the left menu item and design the default tabular view that filters active "Won" quotes versus completed ones.
4. **Step 4: The Gantt Visualizer**
   Integrate your choice of library or build a clean CSS Grid/SVG timeline row. Link the visual chart directly to the active database state so that shifting a slider dynamically moves downstream tasks on screen.